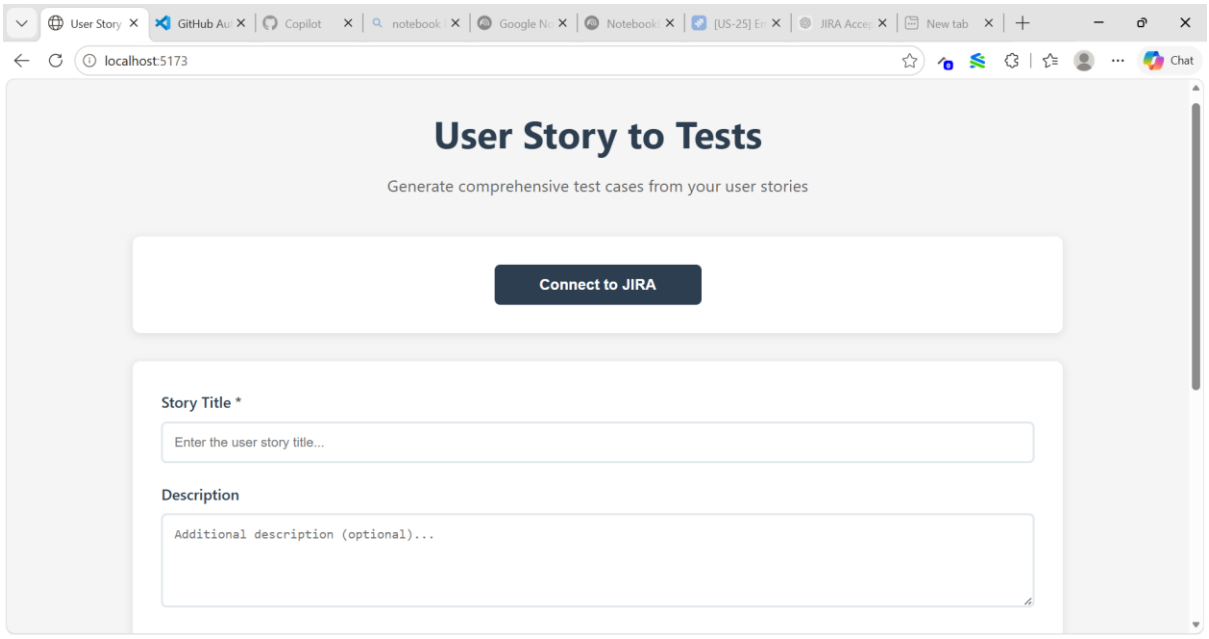
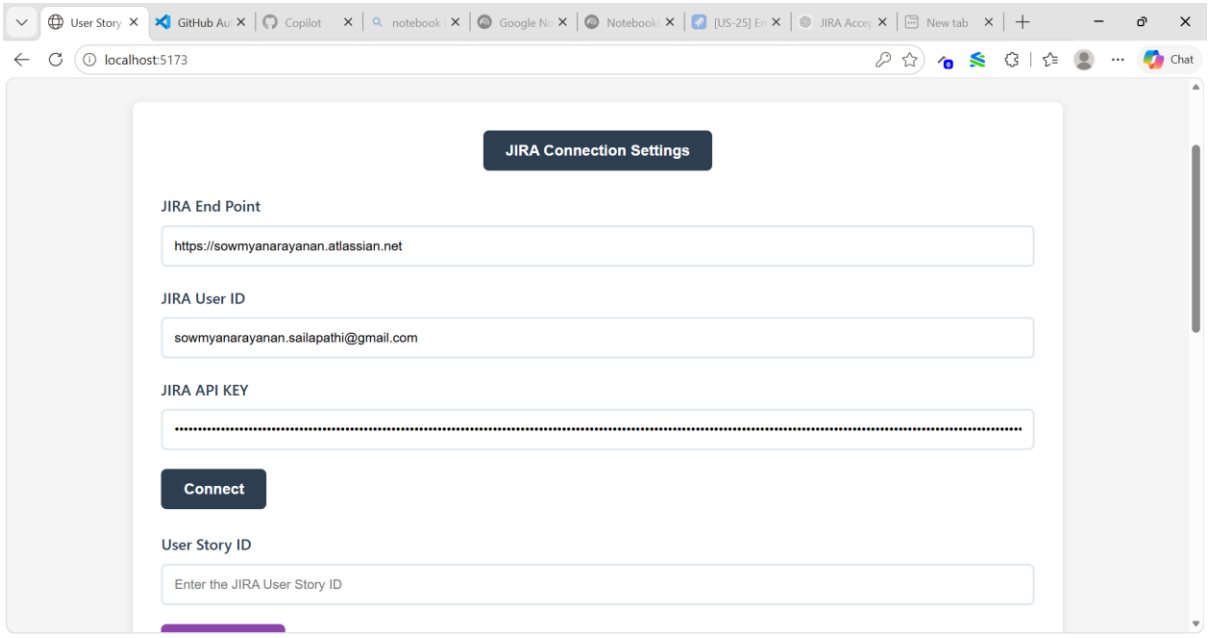
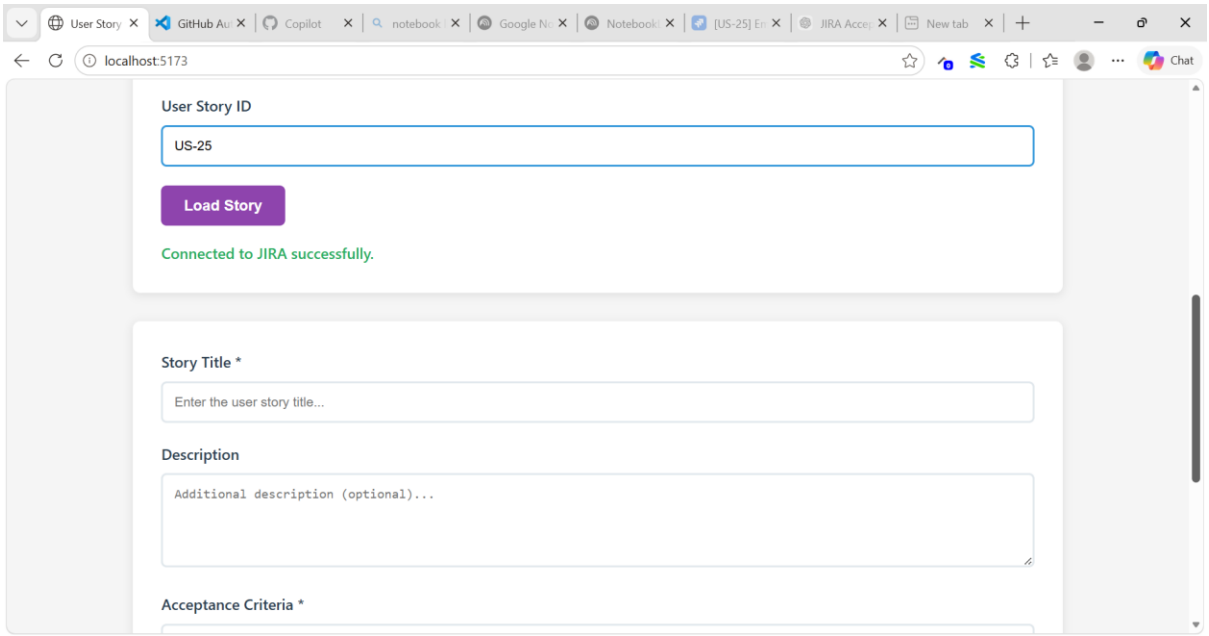
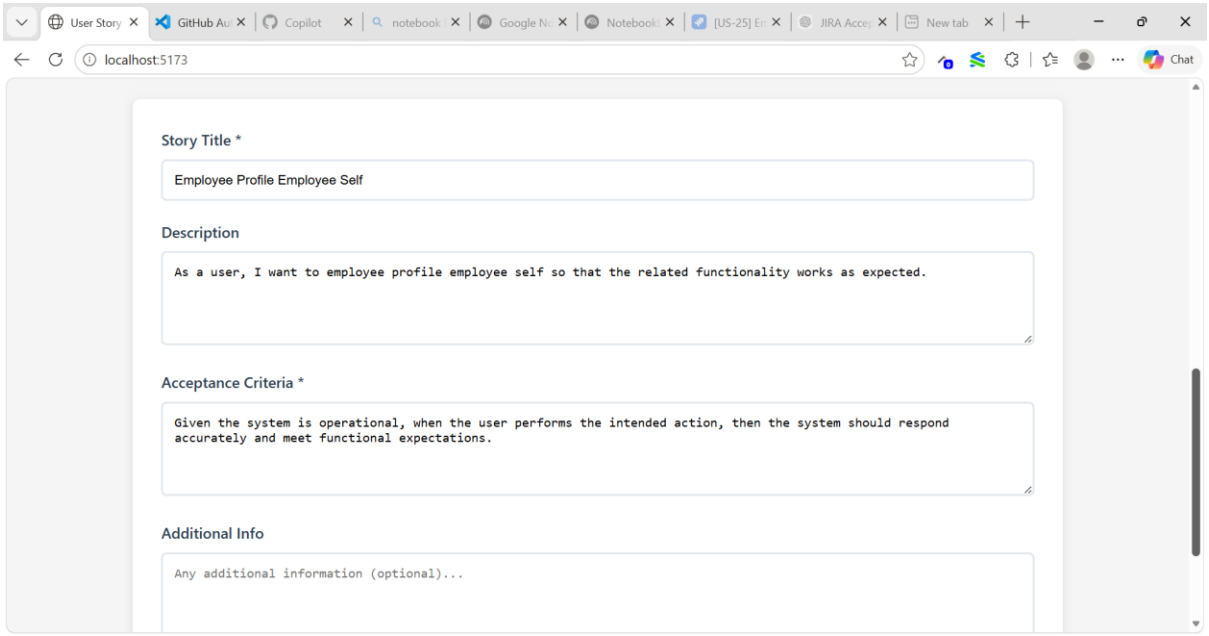
week-3-day-2: user-stories-testcase generator

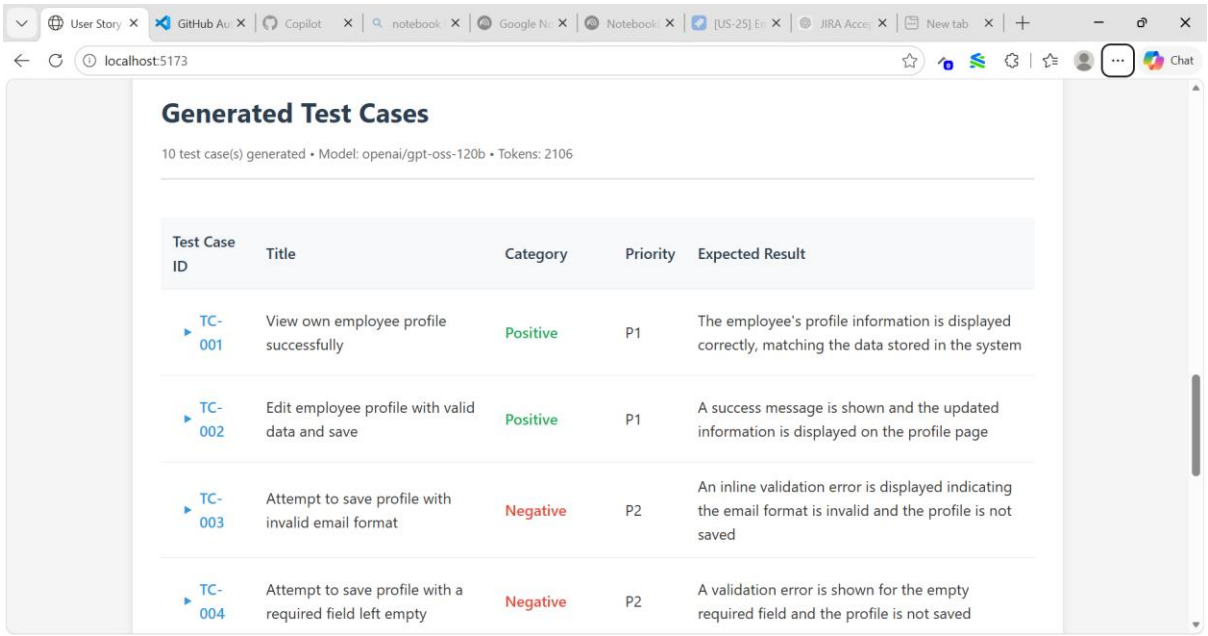
1. **User story -tests generator -- with vibe coding implement the Jira integration to fetch user stories (both frontend and backend)**



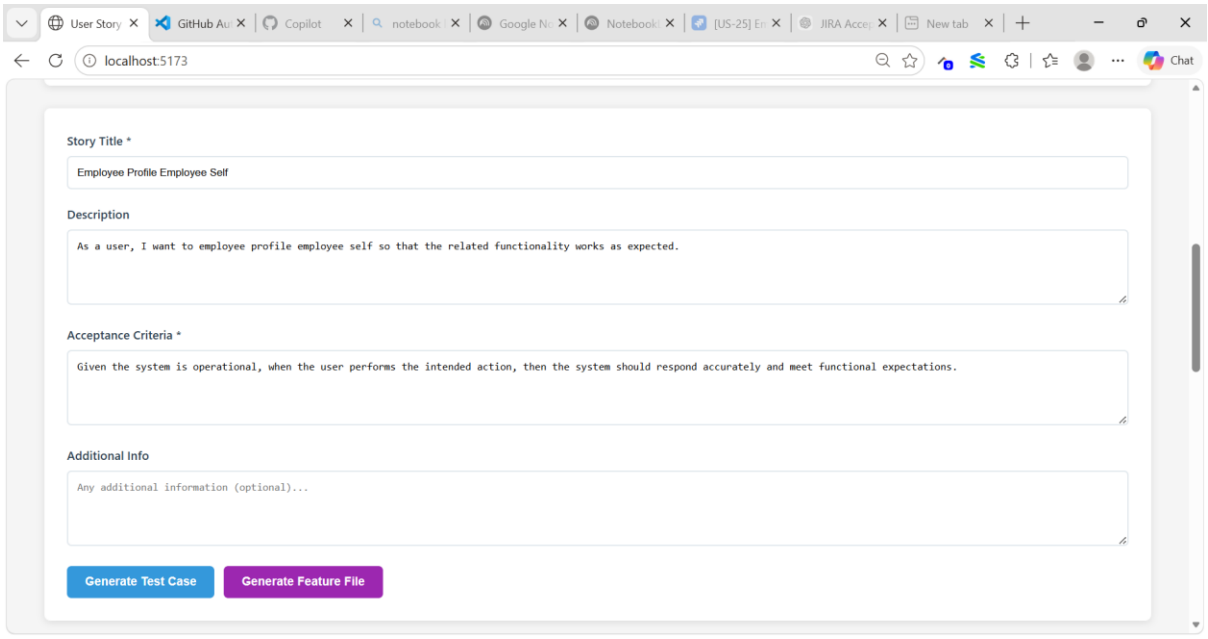


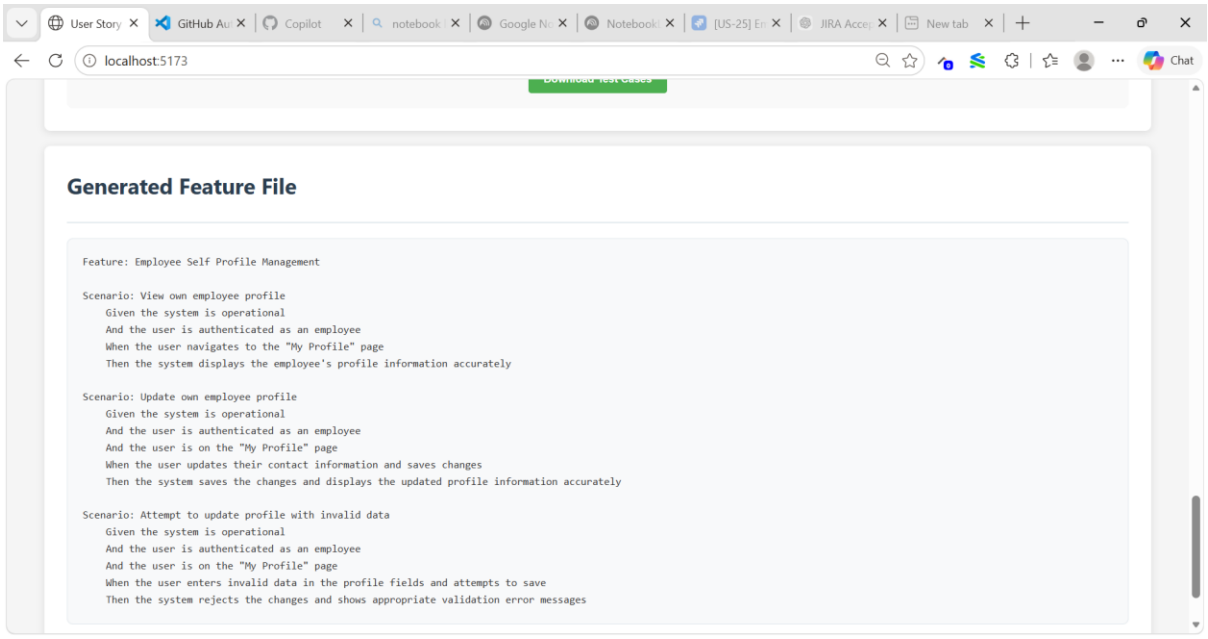

User Story ID
US-25
Load Story
Connected to JIRA successfully.
Story Title *
Enter the user story title...
Description
Additional description (optional)...
Acceptance Criteria *


Story Title *
Employee Profile Employee Self
Description
As a user, I want to employee profile employee self so that the related functionality works as expected.
Acceptance Criteria *
Given the system is operational, when the user performs the intended action, then the system should respond accurately and meet functional expectations.
Additional Info
Any additional information (optional)...



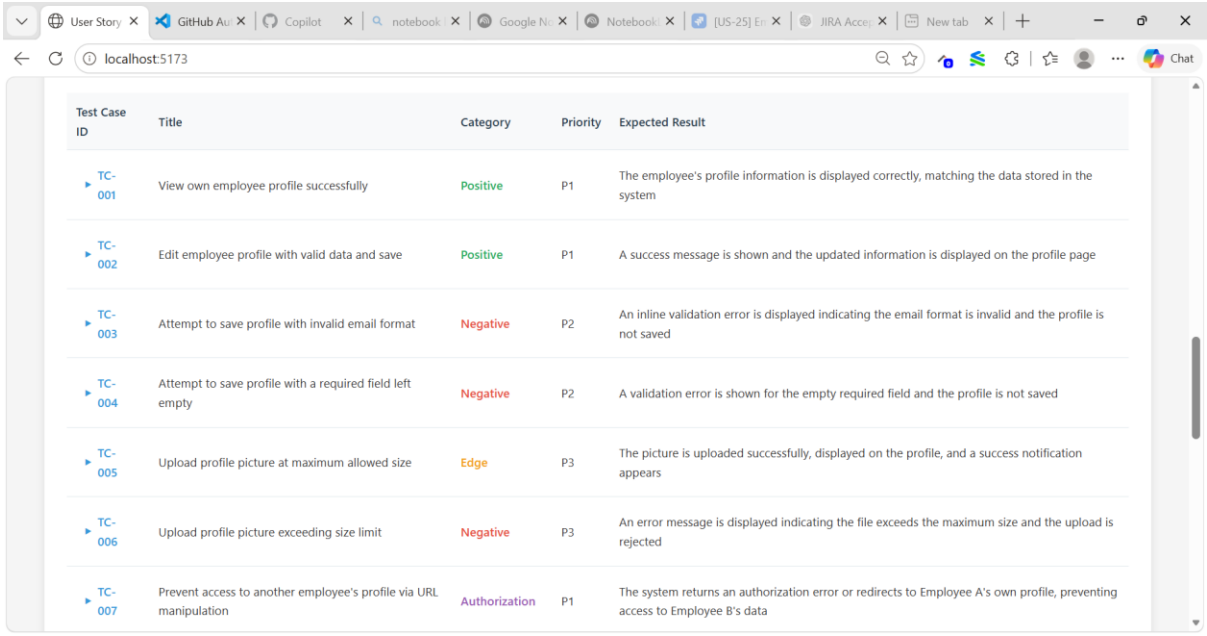
2. **Application is User story - tests generator, Add a button option to Generate Feature file for the user stories**





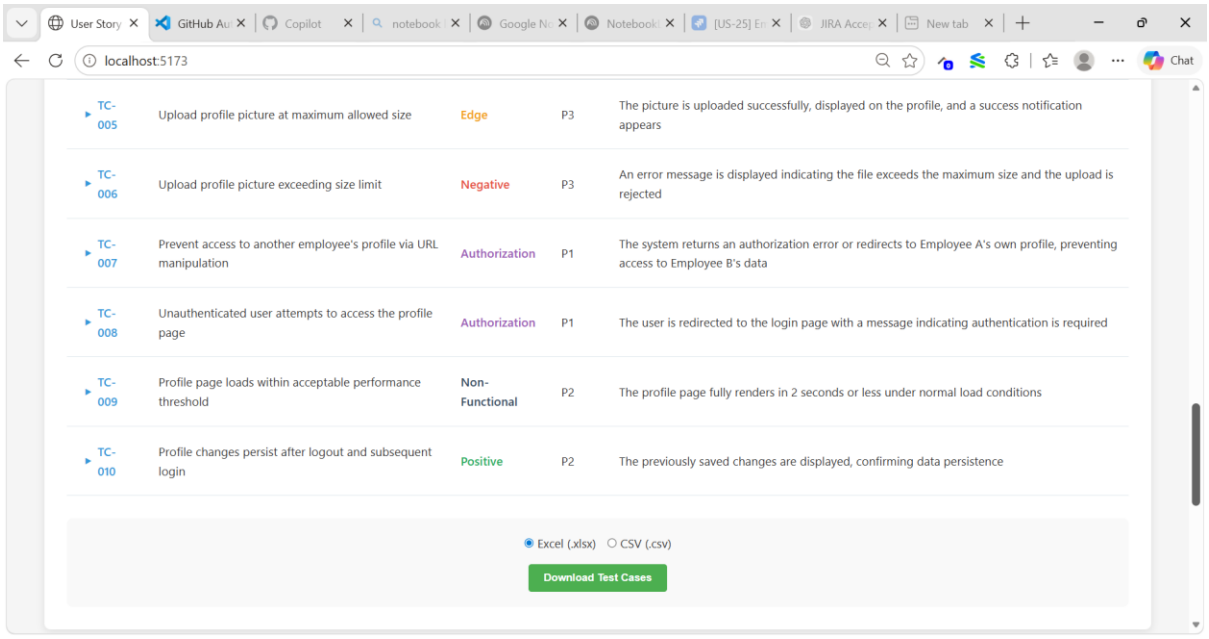
Additional Features:

A. Include Priority Column for Test Cases:



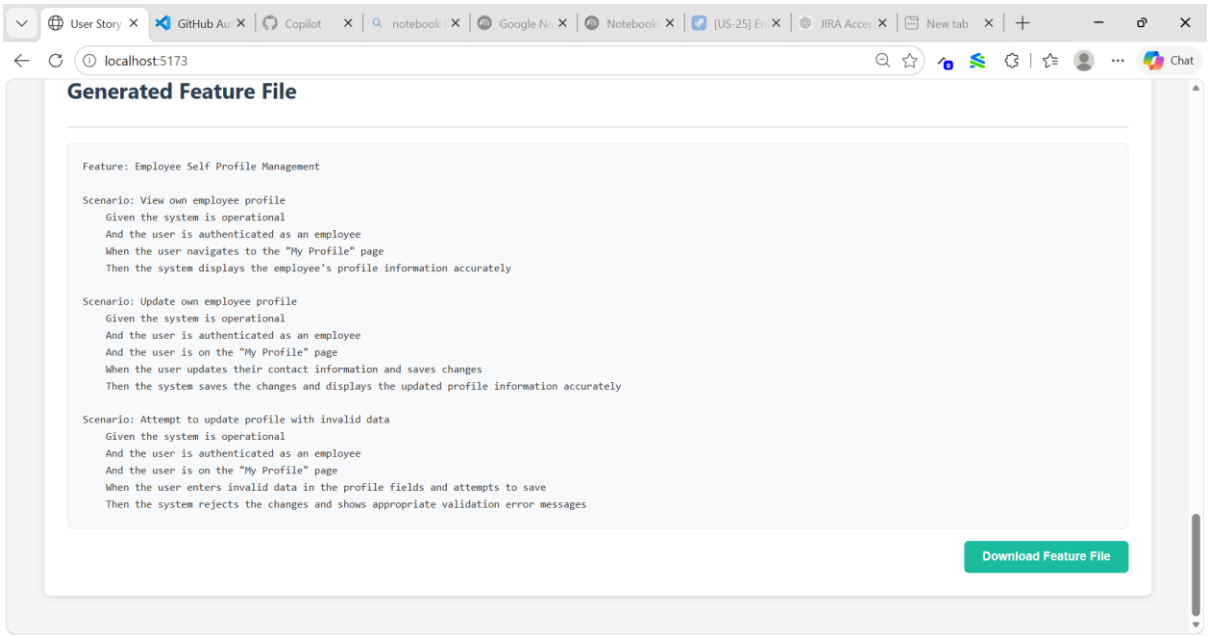
Test Case ID	Title	Category	Priority	Expected Result
TC-001	View own employee profile successfully	Positive	P1	The employee's profile information is displayed correctly, matching the data stored in the system
TC-002	Edit employee profile with valid data and save	Positive	P1	A success message is shown and the updated information is displayed on the profile page
TC-003	Attempt to save profile with invalid email format	Negative	P2	An inline validation error is displayed indicating the email format is invalid and the profile is not saved
TC-004	Attempt to save profile with a required field left empty	Negative	P2	A validation error is shown for the empty required field and the profile is not saved
TC-005	Upload profile picture at maximum allowed size	Edge	P3	The picture is uploaded successfully, displayed on the profile, and a success notification appears
TC-006	Upload profile picture exceeding size limit	Negative	P3	An error message is displayed indicating the file exceeds the maximum size and the upload is rejected
TC-007	Prevent access to another employee's profile via URL manipulation	Authorization	P1	The system returns an authorization error or redirects to Employee A's own profile, preventing access to Employee B's data

B. Download Test Cases in .xlsx and .csv format:



test-cases.csv

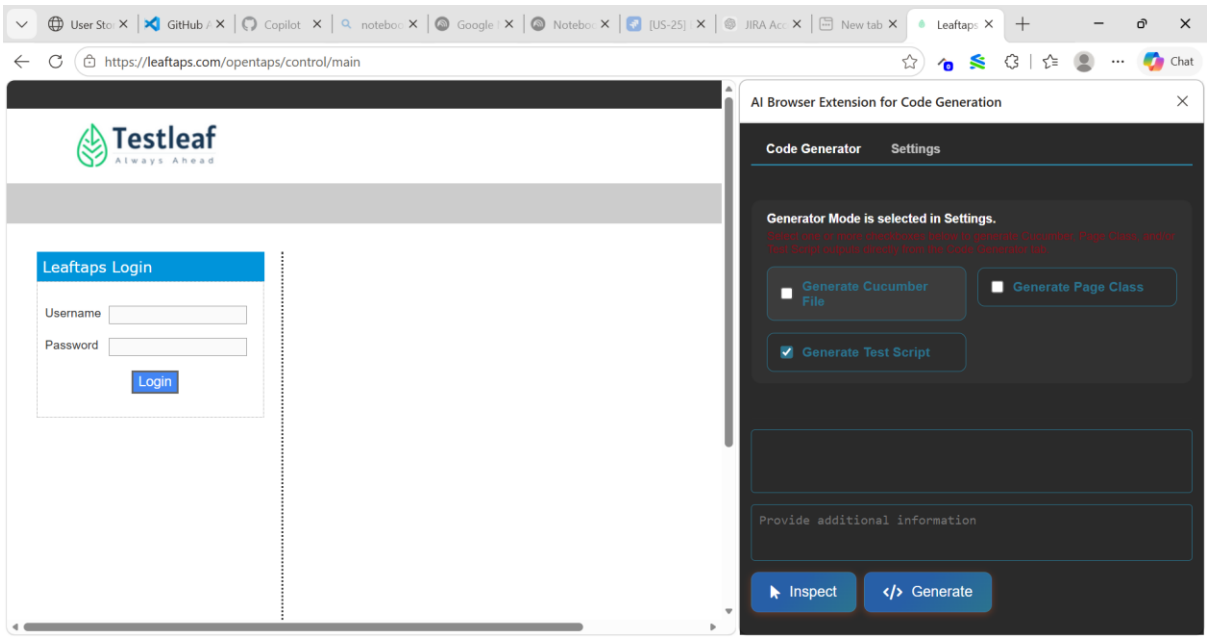
C. Download Feature File with extension. Feature

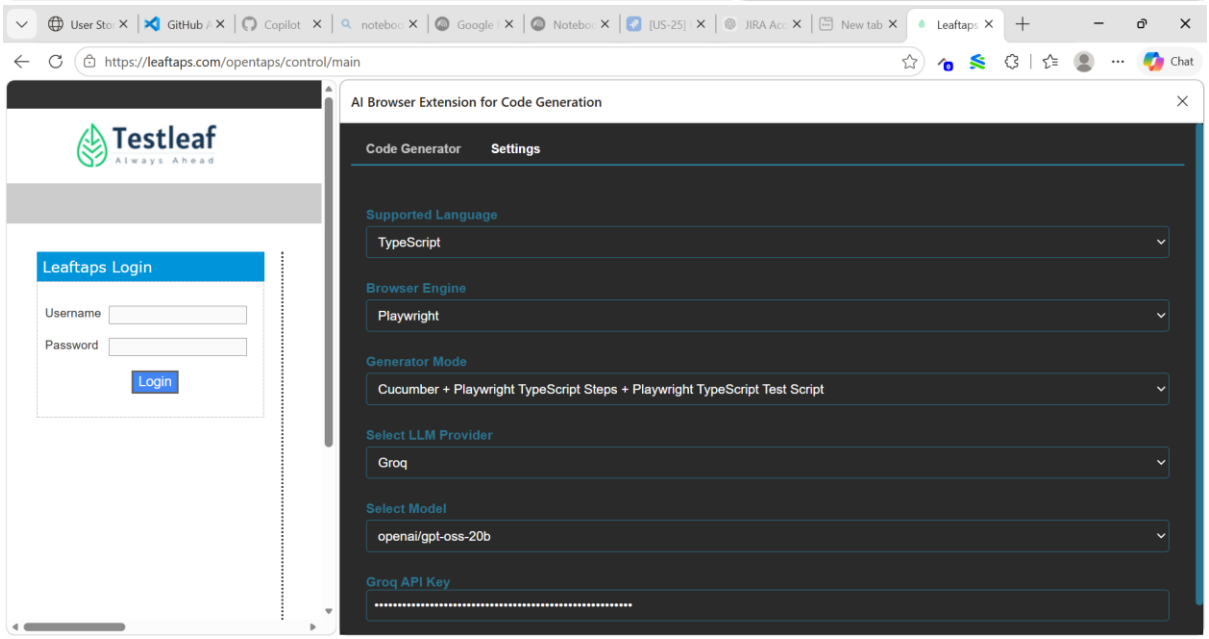


Employee-Profile-Employee-Self.feature

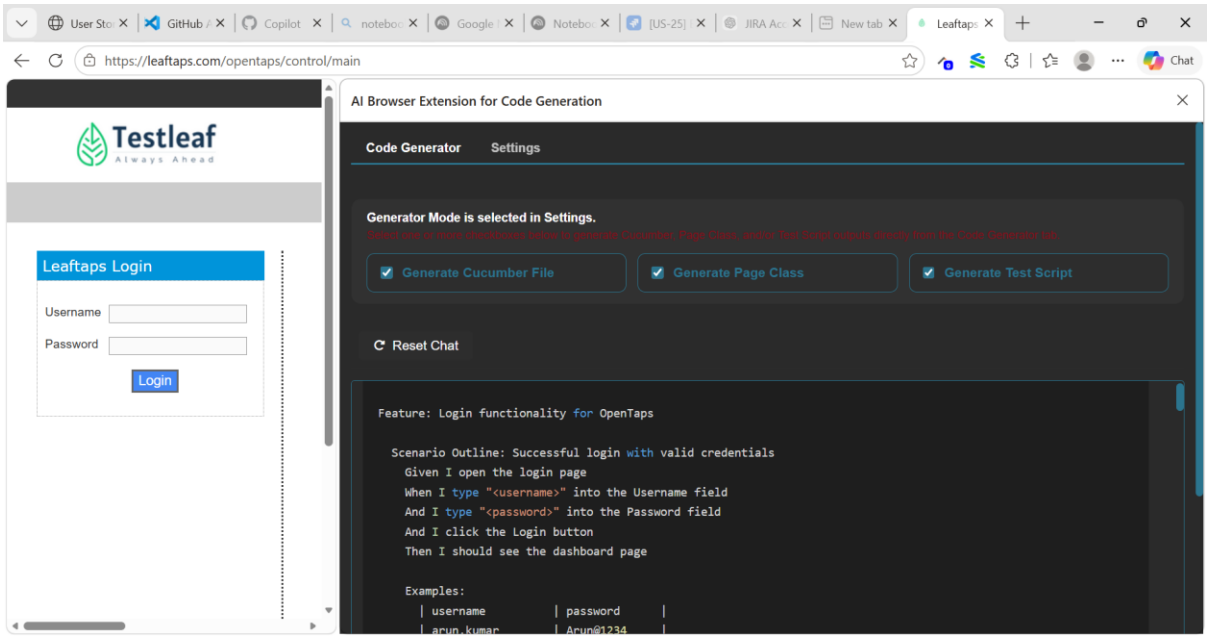
3. **AI Browser Extension - With vibe coding generate the test scripts in the language selected (tool/language of your choice)**

Playwright + Type Script:

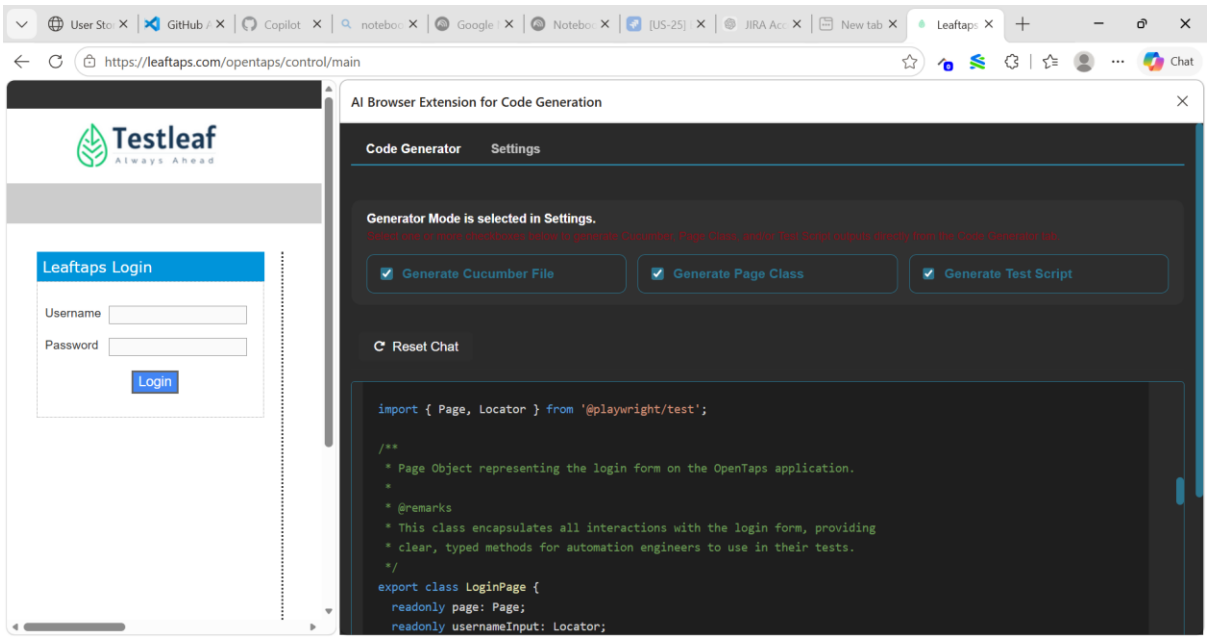




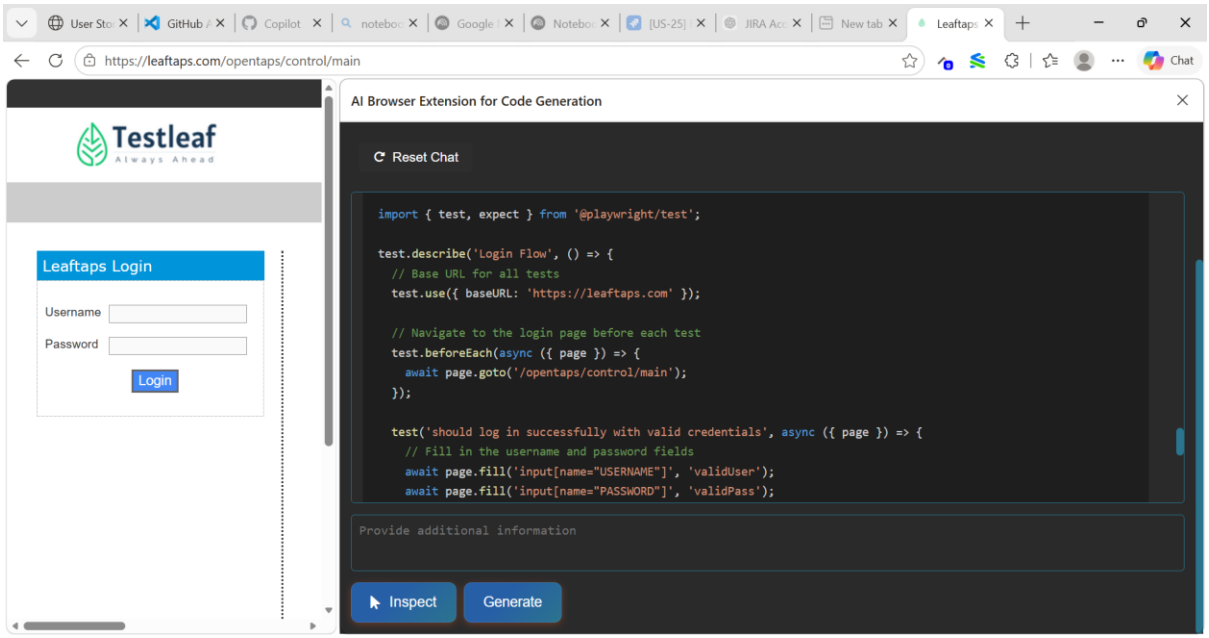
Feature File:



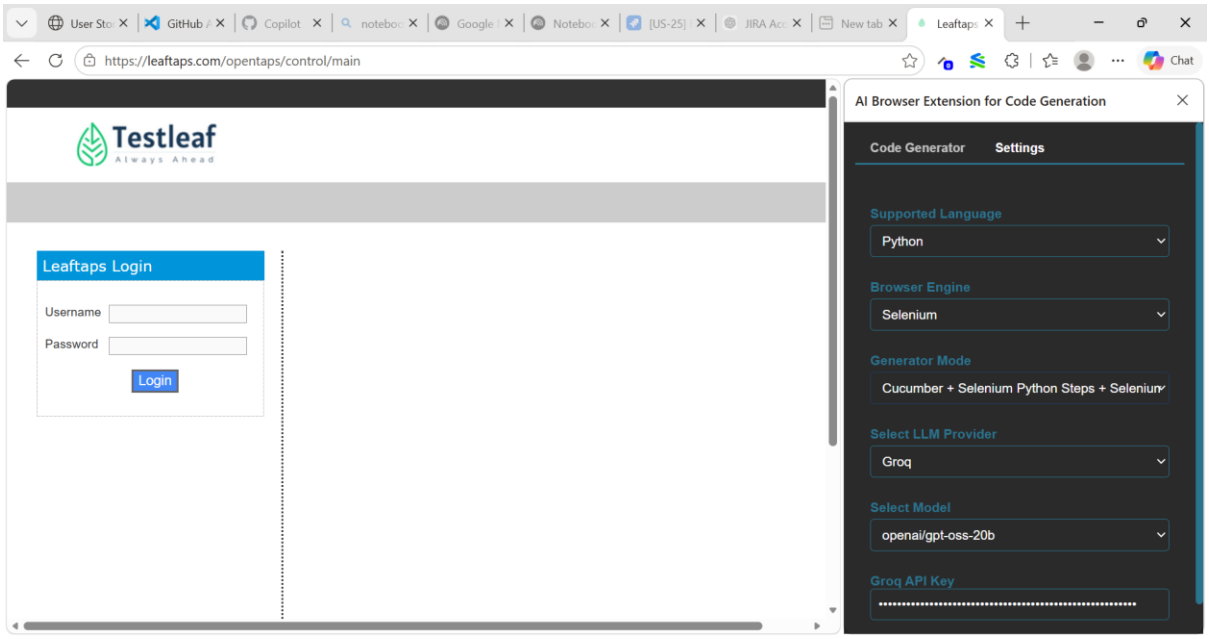
Page Class:



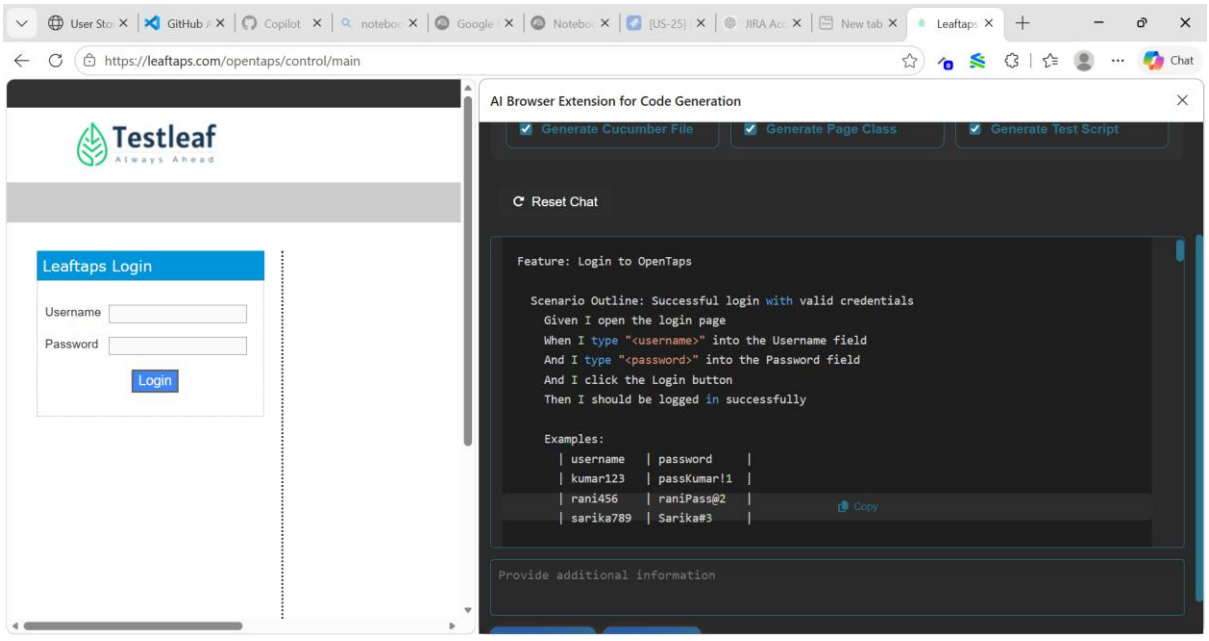
Test Script:


https://leaftaps.com/opentaps/control/main
Chat
Testleaf
Always Ahead
Leaftaps Login
Username
Password
Login
AI Browser Extension for Code Generation
Reset Chat
import { test, expect } from '@playwright/test';
test.describe('Login Flow', () => {
// Base URL for all tests
test.use({ baseURL: 'https://leaftaps.com' });
// Navigate to the login page before each test
test.beforeEach(async ({ page }) => {
await page.goto('/opentaps/control/main');
});
test('should log in successfully with valid credentials', async ({ page }) => {
// Fill in the username and password fields
await page.fill('input[name="USERNAME"]', 'validUser');
await page.fill('input[name="PASSWORD"]', 'validPass');
Provide additional information
Inspect
Generate

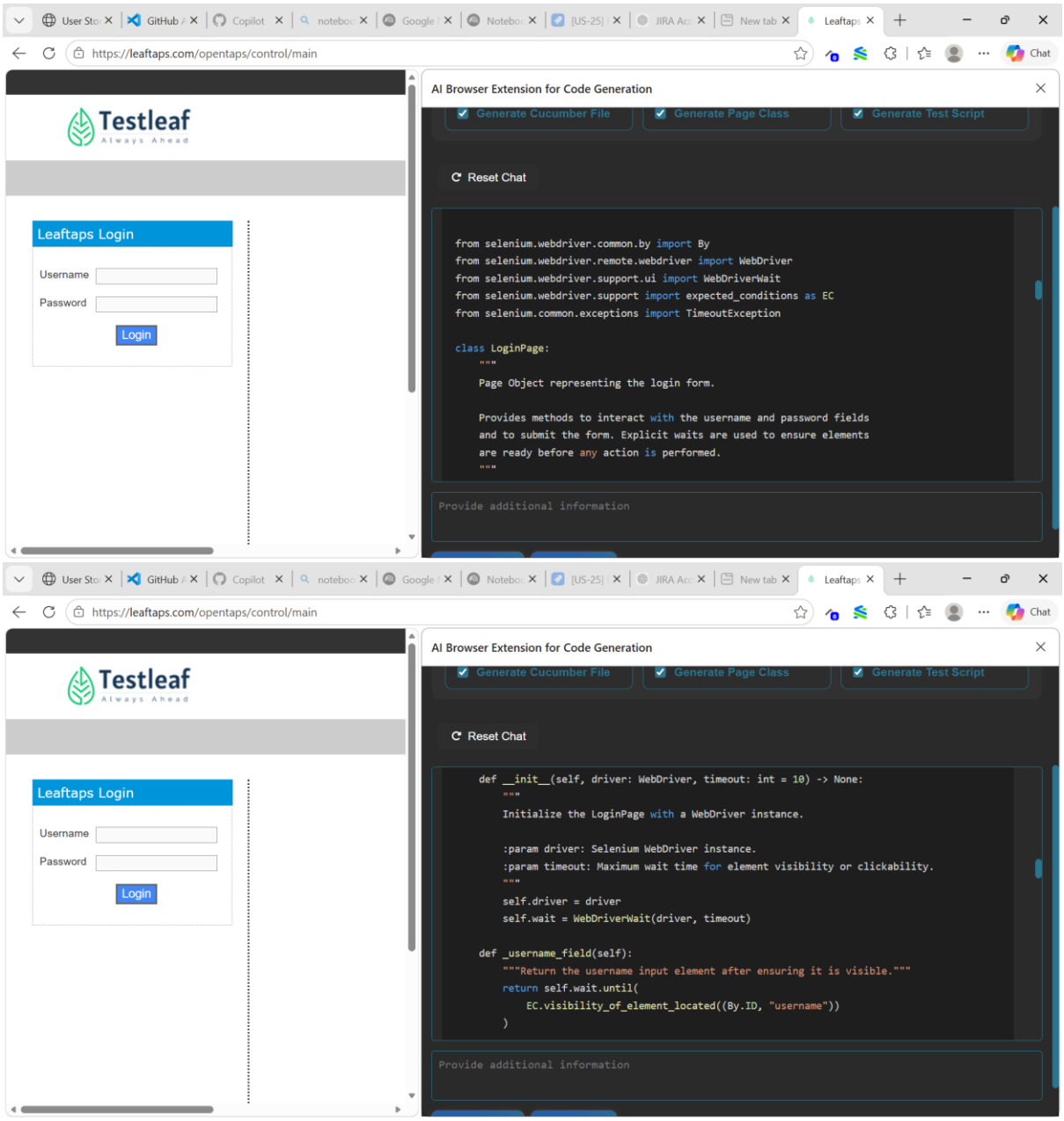
Selenium + Python:



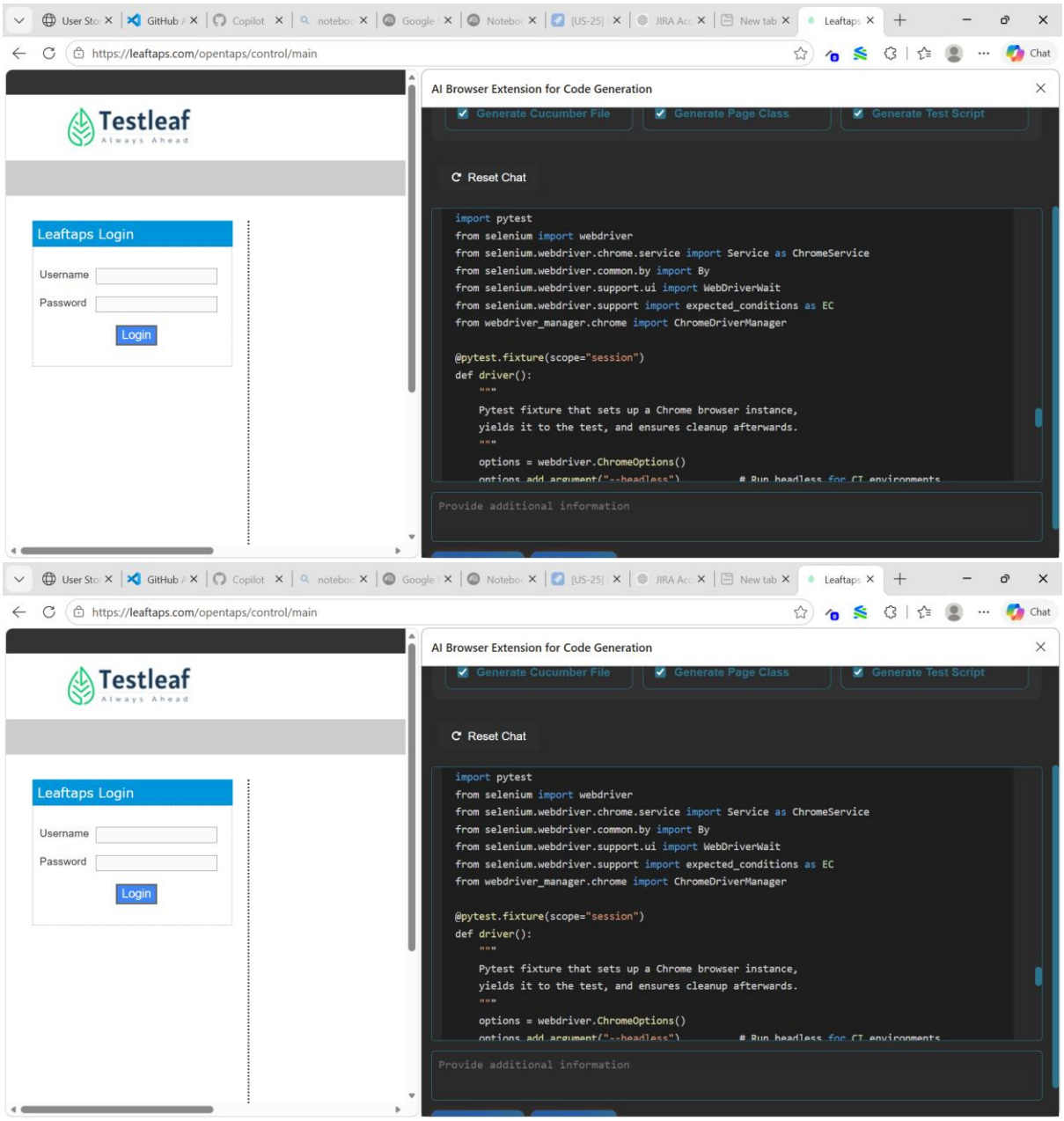
Feature File:



Page Class:


https://leaftaps.com/opentaps/control/main
Testleaf
Always Ahead
Leaftaps Login
Username
Password
Login
AI Browser Extension for Code Generation
Generate Cucumber File
Generate Page Class
Generate Test Script
Reset Chat
from selenium.webdriver.common.by import By
from selenium.webdriver.remote.webdriver import WebDriver
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from selenium.common.exceptions import TimeoutException
class LoginPage:
"""
Page Object representing the login form.
Provides methods to interact with the username and password fields
and to submit the form. Explicit waits are used to ensure elements
are ready before any action is performed.
"""
Provide additional information
https://leaftaps.com/opentaps/control/main
Testleaf
Always Ahead
Leaftaps Login
Username
Password
Login
AI Browser Extension for Code Generation
Generate Cucumber File
Generate Page Class
Generate Test Script
Reset Chat
def __init__(self, driver: WebDriver, timeout: int = 10) -> None:
"""
Initialize the LoginPage with a WebDriver instance.
:param driver: Selenium WebDriver instance.
:param timeout: Maximum wait time for element visibility or clickability.
"""
self.driver = driver
self.wait = WebDriverWait(driver, timeout)
def _username_field(self):
"""Return the username input element after ensuring it is visible."""
return self.wait.until(
EC.visibility_of_element_located((By.ID, "username"))
)
Provide additional information

Test Script:



https://leaftaps.com/opentaps/control/main
Testleaf
Always Ahead
Leaftaps Login
Username
Password
Login
AI Browser Extension for Code Generation
Generate Cucumber File
Generate Page Class
Generate Test Script
Reset Chat
import pytest
from selenium import webdriver
from selenium.webdriver.chrome.service import Service as ChromeService
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC
from webdriver_manager.chrome import ChromeDriverManager
@pytest.fixture(scope="session")
def driver():
"""
Pytest fixture that sets up a Chrome browser instance,
yields it to the test, and ensures cleanup afterwards.
"""
options = webdriver.ChromeOptions()
Provide additional information